📘 Introduction

The mapp Framework 6 User Manual is available online at github https://br-automation-community.github.io/mapp-Framework-6/

To get started with the mapp Framework 6, proceed to the General information section.

> IMPORTANT: The steps on the **"Required Modifications"** page must be executed in order to get the Framework into a functional state!

ℹ️ mapp Framework 6

This section contains information that is relevant for all of the mapp Frameworks.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

ℹ️ Introduction

mapp Technology is the overarching term for the ready-made, modular software products at B&R. This technology enables you to implement complex or tedious features (such as a recipe system) with just a few mapp function blocks and configuration settings rather than creating it from scratch with PLCopen.

By using mapp Technology you can complete your application development up to three times faster, with significantly less code than if you wrote it entirely with PLCopen.

The **mapp Framework 6** takes mapp Technology one step further to provide the user with a universal starting point for mapp Technology. This even further reduces the amount of application code that must be written by the application engineer.

The Framework includes programming tasks and supporting configuration files with built-in best practices and application know-how. It is designed to be modular, so the user can easily add the specific parts that are relevant to the machine to an existing project.

Motivation and Goals

The overall goal of the **mapp Framework 6** is to streamline and simplify your mapp Services and Axis implementation.

More specifically, the goals are:

Quality - The Framework is designed with best practices in mind, which have been vetted by several experienced application engineers - The goal is to set each mapp user up for success

Time Savings - By giving users a reliable starting point, the learning curve of mapp Technology is reduced - New engineers can ramp up faster - Quicker time to market

Simplicity - The **mapp Framework 6** is modular without being overly complex - With a standardized approach to mapp Technology, application support, hand-off, and code maintenance become more straightforward - The framework is scalable according to the needs of the application

Cost Savings - Use of the Framework results in cost savings for the machine due to reduced application engineering time

---

ℹ️ Community Driven Resource

The **mapp Framework 6** is a community driven and open source resource. Input from the community is used to continually refine and improve the **mapp Framework 6**. It is not an official product from B&R.

The GitHub repository that holds the Framework source project is available at: link

Application engineers can suggest modifications or new functionality directly via GitHub. In this way, the **mapp Framework 6** is constantly evolving and improving through community input.

The **mapp Framework 6** uses standard mapp components, and general questions about mapp should continue to be directed to local B&R support.

Because the Framework itself is community driven, questions related specifically to the **mapp Framework 6** should be asked in the B&R Community forum - B&R Community - **mapp Framework 6**

This project is **not part of official B&R R&D development** and is **not covered by official B&R support**, maintenance contracts, or product roadmaps.

ℹ️ Supported Versions

The **mapp Framework 6** is supported in the following versions.

mapp Technologies and Automation Studio

mapp Technologies	Automation Runtime	Automation Studio
6.5 or later	6.5 or later	6.5 or later

Note that the framework uses some retained variables.

ℹ️ Prerequisites

It is expected that the user has a base knowledge of Automation Studio and mapp Technology before utilizing the **mapp Framework 6**. Therefore, the following training courses are recommended as prerequisites:

- SEM210 - Automation Studio
- SEM270 - mapp Services
- SEM611 - mapp View
- SEM415 - mapp Axis

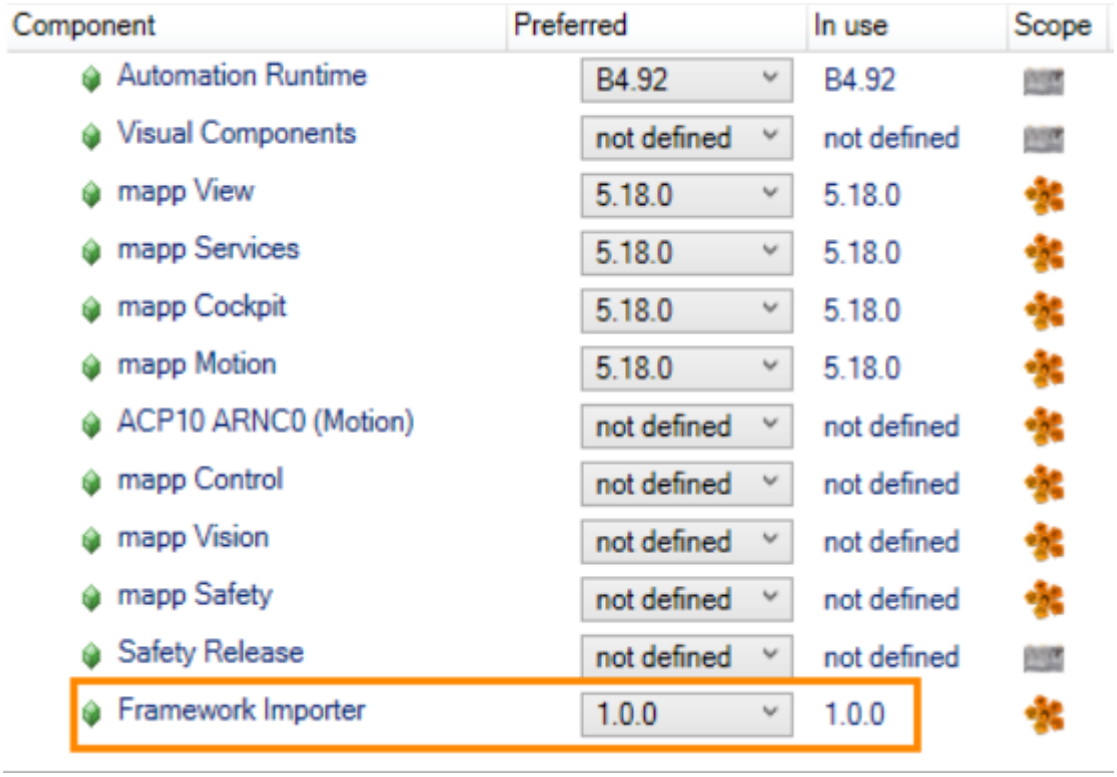
The **mapp Framework 6** is designed and intended to be used by B&R/partner application engineers and customer application engineers alike. In other words, any mapp Technology user can utilize the **mapp Framework 6**.

ℹ️ Launching the importer

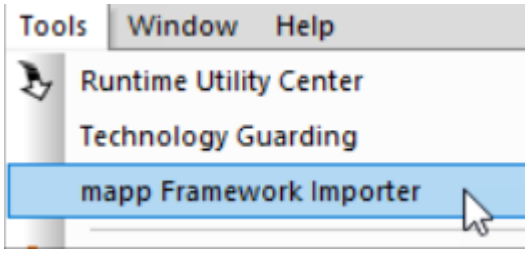
> IMPORTANT:
>
> After installing the **mapp Framework 6**, close and re-open Automation Studio before you try to launch the importer.

You **must** close the software configuration and CPU configuration before performing an import.

To launch the **mapp Framework 6** import tool within Automation Studio, first make sure to set a version for the Framework in Project -> Change Runtime Versions:



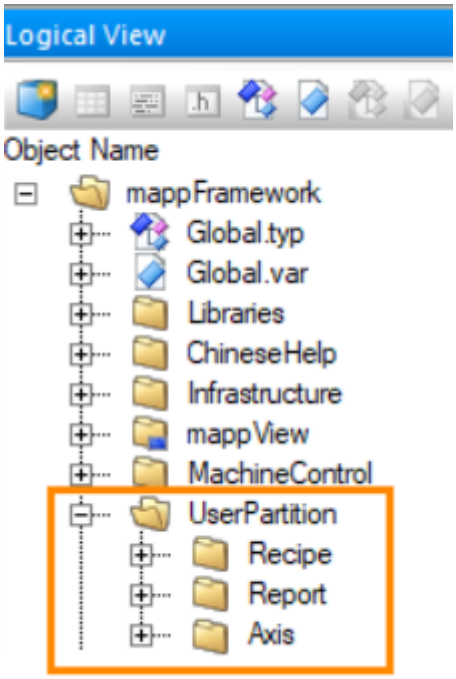
Then navigate to Tools → **mapp Framework 6** Importer:



ℹ️ Default User Partition Files

The Framework contains a few default files (e.g. recipe files) that need to be transferred to the user partition via an offline install or FTP access. These files are located in the Logical View within the "UserPartition" package.

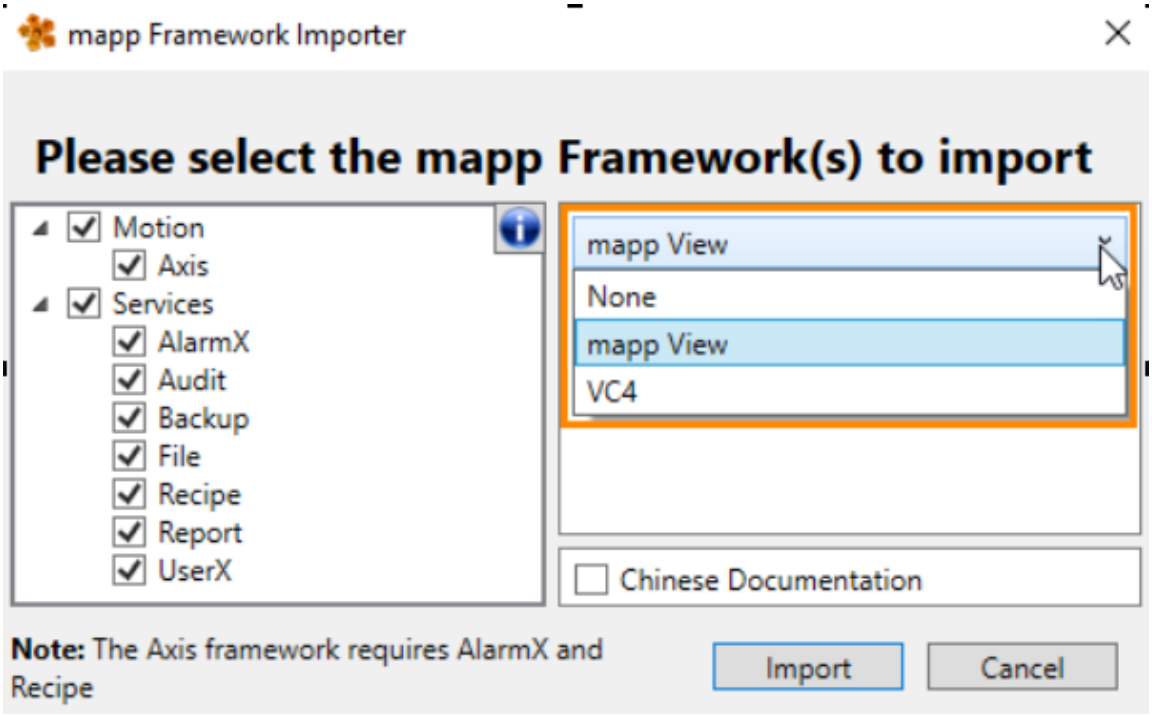
(If the user changed the **mapp Framework 6** file devices to correspond to a location other than the user partition, then these files need to be placed in that new location instead.)



ℹ️ Visualization - mappView

The **mapp Framework 6** offers a mapp View front end.Select your preference via the dropdown in the import tool.You can also select to import no visualization and just get the backend code.

The mapp View visualization supports a modular import.You will only get the visualization files for the framework components that you choose to import.



Interface to the HMI

Each **mapp Framework 6** has a structure variable for **commands, parameters, and status information** from the HMI.

This variable name always starts with Hmi followed by the mapp Technology.
Example: HmiRecipe

> Note:
> The only exception is the Axis framework, where the HMI is linked directly to the AxisCommands structure.

Version Information

This section describes the new features in each version of the **mapp Framework 6**. The **mapp Framework 6** versioning system follows the versioning scheme of mapp Technology:

> **Major.Minor.Bugfix**

Topics in this section:

- Version 6.5

v6.0.0 - Migration from AS4 to AS6

Framework / Component	Description
mapp AlarmX	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
mapp Axis / Cockpit	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
mapp Backup	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
mapp File	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
mapp Recipe	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
mapp UserX	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
mapp Report	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
Import Tool	**Key changes:** migration to AS6 **Bugfixes:** - **New behavior:** -
Documentation	

mapp AlarmX Framework 6

This section describes the mapp AlarmX Framework 6. For full details on mapp AlarmX itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

- Features
- Task Overview
- Script
- Required Modification
- Optional Modification

mapp AlarmX Framework 6 Features

The following features and functionality are included in the mapp AlarmX Framework 6:

- 100 ready-made discrete value monitoring alarms and a Boolean array to trigger each alarm
- Localizable text for each alarm
- Alarm mapping by severity with reactions
- A query along with the supporting state machine to query large amounts of data
- mapp View content to display current alarms, alarm history, and the alarm query
- The ability to acknowledge and export alarms from the HMI

Embedded Examples

The following examples are embedded into the Framework:

- Examples for each type of monitoring alarm.Details are provided in the comments in the **AlarmSamples.st** action file, starting on line 5.
- Alarm names:
  - LevelMonitoringExample
  - DeviationMonitoringExample
  - RateOfChangeExample
- Example for incorporating a snippet into an alarm. This example is provided to allow easy copy and paste of the syntax for referencing a snippet in the mapp AlarmX configuration.
- Alarm name: SnippetExample
- Example of using MpAlarmXAlarmControl to manually set and reset an alarm from code
- Alarm name: MpAlarmXControlExample
- The supporting code is shown in the **AlarmSamples.st** action file on lines 24–32

AlarmMgr Task Overview

The **AlarmMgr** task contains all of the provided Framework code for mapp AlarmX.

This task is located in the Logical View within the Infrastructure package.
The chart below provides a description of the main task and each action file.

The **AlarmMgr** task is deployed to Task Class 1 by the Import Tool.

AlarmMgr Task Files

Filename	Type	Description
AlarmMgr.st	Main task code	General mapp function block handling. Handles alarm acknowledgment. Handles alarm history export. Checks if any reactions are active. Calls all actions.
AlarmHandling.st	Action	Defines the conditions which trigger each alarm. The AlarmMgr task contains a Boolean array called Alarms. Each index of the array corresponds to the Monitored PV for each of the 100 predefined alarms in the AlarmX configuration. This is also the designated place to define the conditions under which alarms should be inhibited, if inhibiting is required in the application.
HMIActions.st	Action	Shows the alarm backtrace information on the HMI. Typically, the backtrace action (GetBacktraceInformation) does not need to be modified. Also sets up the table configuration for the query table.
ExecuteQuery.st	Action	Executes a query. Includes the supporting state machine used to query large amounts of data.

AlarmX Configuration Import and Export via Python

A Python script is provided within the AlarmX package in the Logical View.

This script allows you to export the AlarmX configuration to Excel and then re-import the AlarmX configuration back into the project after you've made any necessary changes.

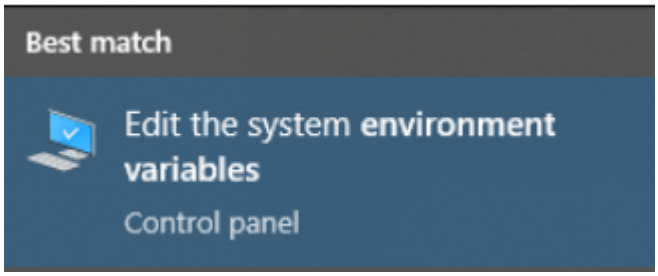
Prerequisites for Using the Script

1.Download and Install Python

-> Download and Install Python

2.Edit Your System Path

1.From the Control Panel, navigate to the advanced system settings. Alternatively, type "environment variables" in the start menu search bar and select "Edit the system environment variables".



2.Click on "Environment Variables".

Required Modifications

The Framework provides a solid foundation by default.
However, to fully integrate it into an application, several modifications are required.

The following steps must be completed to bring the Framework into a functional state within the application.

> IMPORTANT:
> The steps on this page must be executed in order to get the Framework into a functional state.

---

1. Define the condition to trigger each alarm

- The AlarmMgr task contains a Boolean array called Alarms. Each index of the array corresponds to the monitored PV for one of the 100 predefined alarms in the AlarmX configuration.
- For each of these alarms, set the corresponding Alarms[] bit equal to the alarm condition relevant to the application.This is done in the **AlarmHandling.st action** file.

Example:
If Alarms[0] should trigger when the light curtain is interrupted and the system is not in maintenance mode, then the alarm condition must reflect this logic.

Alarms[0] := LightCurtainInterrupted AND NOT MaintentanceMode

---

2. Define the alarm text for each alarm

- Define a unique alarm text for each alarm in the Alarms.tmx file.
- Alarms.tmx is located in the Logical View under the Infrastructure package and the AlarmX package.
- Text ID Alarm.0 corresponds to Alarm0 (Alarms[0]).Text ID Alarm.1 corresponds to Alarm1 (Alarms[1]), and so on.
- Define the alarm text for all languages relevant to the application.

Optional Framework Modifications

The Framework can be adjusted as needed for the application in any way necessary.

This section summarizes some optional modifications that are commonly done to the Framework.

- Adjust the provided query (ActiveAlarms) source conditions (SELECT and WHERE) in AlarmXCfg.mpalarmxcore.
- Decide how to handle the situation where the query result contains more than 20 alarms. Refer to the comments in ExecuteQuery.st starting on line 38.
- If alarms are triggered via MpAlarmXControl or MpAlarmXSet, consider setting and resetting alarms throughout the application and not solely within the AlarmMgr task.
    a. For example, if an error occurs in the recipe system, the alarm can be triggered directly in the recipe task.
    b. Decentralization in this way is a key benefit of mapp AlarmX.
- By default, the provided query runs only when the Run query button is clicked on the HMI. If you want the query to run whenever new data is available, refer to the comments in the ACTIVE_ALARM_WAIT state of the ExecuteQuery action.
- In the CPU configuration, modify the mappAlarmXFiles file device to the desired storage medium. By default, this corresponds to the User partition (F:\AlarmX).
    a. If this is changed, update or delete lines 10–16 of the AlarmMgr.st INIT program, which create the F:\AlarmX directory if it does not already exist.
- Add additional alarms or delete unused alarms (see here for more details).
- Adjust the alarm mapping (see here for more details).
- Inhibit specific alarms under defined conditions (see here for more details).
- Add additional queries (see here for more details).

Topics in this section:

- Add or Delete Alarms
- Alarm Mapping
- Inhibit Alarms
- Add Queries

Adding Alarms

The Framework comes with **100 discrete value** monitoring alarms. If you need more discrete value monitoring alarms or any other type of alarm, add them accordingly.

The steps are as follows:

1. Increase the size of the Alarms[] array in AlarmMgr.var according to how many alarms you need to add.
2. Add the new alarms to the Alarm List in the AlarmXCfg.mpalarmxcore configuration file. Set the monitored PVs to the newly added elements of the Alarms[] array from step 1.
3. Define the condition to trigger each new alarm in the AlarmHandling.st action file.

> Note that it is permissible to mix and match different types of alarms. For example, monitoring alarms can be combined with edge or persistent alarms that are triggered via MpAlarmXAlarmControl or MpAlarmXSet.

Deleting Alarms

The Framework comes with 100 discrete value monitoring alarms. If you do not need all 100 alarms, delete them as required.

The steps are as follows:

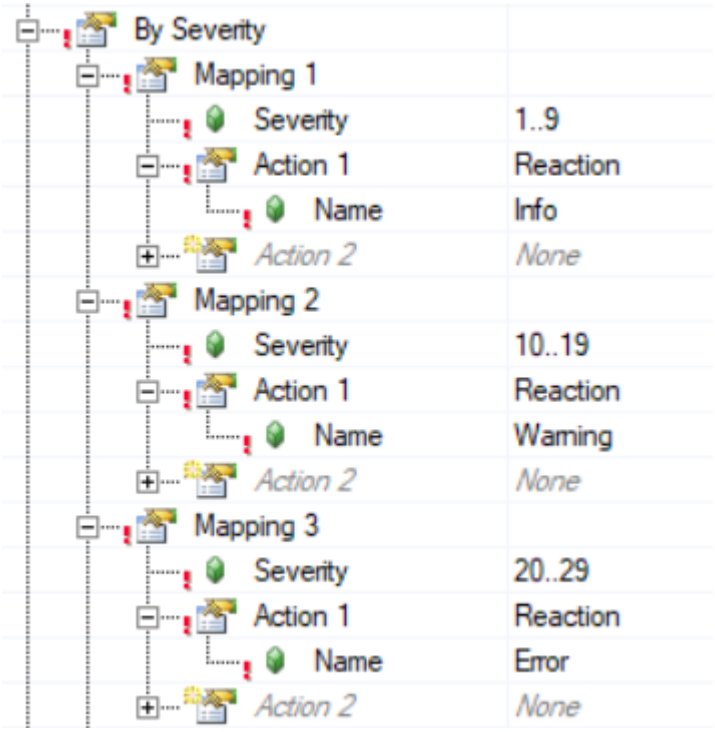
1. Optionally decrease the size of the Alarms[] array in AlarmMgr.var according to how many alarms you want to remove.
2. Delete the unused alarms from the Alarm List in the AlarmXCfg.mpalarmxcore configuration file.
3. Delete the alarm assignments for the unused alarms in the AlarmHandling.st action file.

Note that these assignments may already be commented out from the initial Framework import.

Alarm Reactions

The Framework establishes three alarm reactions by default within the alarm mapping.

1. Info (severity 1–9)
2. Warning (severity 10–19)
3. Error (severity 20–29)



The **MpAlarmXCheckReaction()** function is called for each reaction within the **AlarmMgr** task.

Inhibiting Monitoring Alarms

Decide whether each monitoring alarm should be inhibited at any point.

If so, proceed as follows:

1. Assign an Inhibit PV to the alarm within the Alarm List of the AlarmXCfg.mpalarmxcore configuration file.
    a. This is an advanced parameter, so remember to click the advanced settings icon.
2. In the **AlarmHandling.st action**, write an IF statement to set the Inhibit PV according to a specific condition.
    a. It is common to use the same Inhibit PV to inhibit multiple alarms.
    b. For example, a maintenance mode bit set to True can simultaneously inhibit alarms that should not be monitored during machine maintenance.

An example of using the Inhibit PV is built into Alarm0.The condition for setting CommissioningModeActive to True is not yet defined in AlarmHandling.st and must be implemented based on the application logic.

Adding Additional Queries

The **ExecuteQuery.st action** file contains the programming required to execute the query that comes with the Framework, the ActivateAlarms query.

It also contains the supporting state machine that is used to query large amounts of data.

If you would like to add an additional query, the following steps are required.

1. AlarmMgr.var

1. Copy and paste the variable declaration for QueryActiveAlarms and give it a unique name. Each query needs a dedicated function block instance.
2. Copy and paste the variable declaration for AlarmQuery and give it a unique name.

---

2. AlarmXCfg.mpalarmxcore configuration file

1. Define the new query in the Data Queries section at the bottom of the configuration file. Give it a unique name.
2. Use the new query variable created in step 1.2 for the process variable connections and update count.

---

3. ExecuteQuery.st

Copy and paste lines 3–47 of ExecuteQuery.st to duplicate the code.Then within the copied code:

1. Replace every instance of QueryActiveAlarms with the new function block name created in step 1.1.
2. Replace the query name assignment with the name of the new query created in step 2.1.
    - NewQueryFUB.Name := ADR('NewQueryNameFromStep2.1');
3. Replace every instance of AlarmQuery with the new variable name created in step 1.2.

🧭 mapp Axis Framework 6

This section describes the **mapp Axis Framework 6**. For full details on **mapp Axis** itself, see here. For full details on **mapp Cockpit** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

- Features
- Task Overview
- Required Modification
- Optional Modification

🧭 mapp Axis Framework 6

The following features and functionality are included in the **mapp Axis Framework 6**:

• **Single-axis implementation** which can be repeated for multiple axes.
Includes basic commands, **manual/automatic modes**, and an overarching **state machine** for axis control

• **Cyclic data exchange** for current, lag error, and motor temperature

• **Detailed alarm integration** with **mapp AlarmX**

• **Automatic setup** of **mapp Cockpit**

• **mapp View content** to show the **axis faceplate**

The **mapp Axis Framework 6** is designed so that the main axis control code within the **AxisTemplate** package is reused for all axes that you add. This makes it easy to update the overall process for **all axes simultaneously**.

Any modifications made to the following files within this package will apply to **all axes**:

• **AxisMgr.var**
• **AxisStateMachine.st**
• **ChangeConfiguration.st**
• **Recipe.st**

The Framework comes with the **AxisTemplate** package as well as **one application axis** set up for you in the **AppAxis_1** package. To add additional axes, see here.

Axis-specific code is written within a **dedicated package** for that axis.For more details, see here.

🔐 Access Rights

The ability to interact with the **axis faceplate** on the **mapp View HMI** is restricted to the **Administrators**, **Service**, and **Operators** roles.

The **AppAxis_1** task contains all of the provided Framework code for **mapp Axis**.
This task is located in the **Logical View** within the **MachineControl** package.

The **AppAxis_1** task is deployed to **Task Class 1** by the **Import Tool**.

📄 Task Files

Filename	Type	Reference vs Unique	Description
AxisMgr.st	Main task code	**Unique / dedicated file per axis**	General **mapp function block** handling. Calls all actions.
AxisStateMachine.st	Action	**Referenced file within the AxisTemplate package**	Generic **state machine** used for all axes. Includes states such as power-on, homing, manual/ automatic operation, stopping, etc.
SimulationControl.st	Action	**Unique / dedicated file per axis**	Supporting code for **simulation**. Modify as needed for simulation requirements.
AxisControlModes.st	Action	**Unique / dedicated file per axis**	Axis-specific programming. Contains **manual and automatic mode state machines**. Manual mode is set up for basic jogging. Automatic mode is empty by default and intended to be filled per application.
Recipe.st	Action	**Referenced file within the**	Registers axis variables to the **recipe system**.

🔧 Required Modifications

The Framework by default provides a solid foundation, but in order to integrate it fully into your application there are a few modifications that must be made.
The following list of modifications are required to get the Framework in a functional state within the application.

> 🚨 **IMPORTANT:**
> The steps on this page must be executed in order to get the Framework into a functional state!

---

1. If you plan to run the axis on **physical hardware** (not via a pure virtual), do the following:

   a. Add the **drive** which will control the axis to the **Physical View**.

   b. In the drive configuration, set the **axis reference** to the **MpLink** from the single axis configuration file.

   c. Delete the **VAppAxis1.purevaxcfg** file.

   d. Change the value of **MC_BR_ProcessConfig_ACP.DataType** from **mcCFG_PURE_V_AX** to **mcCFG_ACP_AX**
   in the **ConfigurationInit** action (line 16 of **ChangeConfiguration.st** in the **AxisTemplate** package).

   If at any point you need to simulate this axis after configuring it to run on real hardware, be sure to set a **version number for McAcpSim** within **Change Runtime Versions** (check the **Advanced** box and then expand **mapp Motion**).

2. Implement **Automatic Mode** for the axis.This is done within the **AxisAutomatic** action of the **AxisControlModes.st** file. After importing the Framework, automatic mode is **empty** and must be programmed according to the needs of the application.

3. Edit the **SimulationControl.st**, **AxisMgr.st**, **ManualCommand.st**, and **AutomaticCommand.st** files according to the unique application requirements of your axis. For details about what these files are intended for, see Task Overview.

4. The **axis task(s)** should run in **cyclic 1**, and the maximum allowed cycle time is **20 ms**. Adjust the **TC1 cycle time** accordingly.

5. Change the passwords for the **Admin**, **Operator**, and **Service_Tech** users.This is done in the **User.user** file in the Configuration View (**AccessAndSecurity** → **UserRoleSystem** → **User.user**). Note that if you already had users in your project with these same names prior

⚙️ Optional Modifications

The Framework can be adjusted as needed for the application in any way necessary.
This section summarizes some optional modifications that are commonly done to the Framework.

• The Framework comes with the **axis template files** and **one axis already set up** (**AppAxis_1**).
To add an additional axis, see here.

• To rename **AppAxis_1** to something that more specifically reflects its purpose, see here.

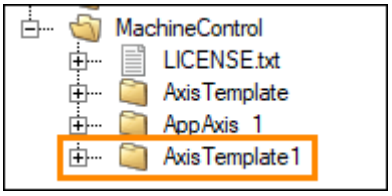
• The default **homing mode** defined in the axis configuration is **Restore Position**.
If the restore position is not valid, then a **direct home** will be executed instead.
If you prefer to execute a different homing mode in this case, then change line **77** of **AxisStateMachine.st** from **mcHOMING_DIRECT** to the desired mode.

Topics in this section:

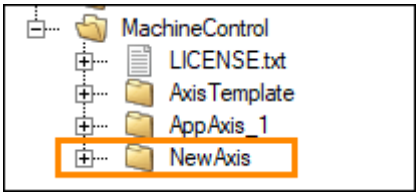
Adding additional axis

Here are the steps to add a new axis to the project using the provided axis template:

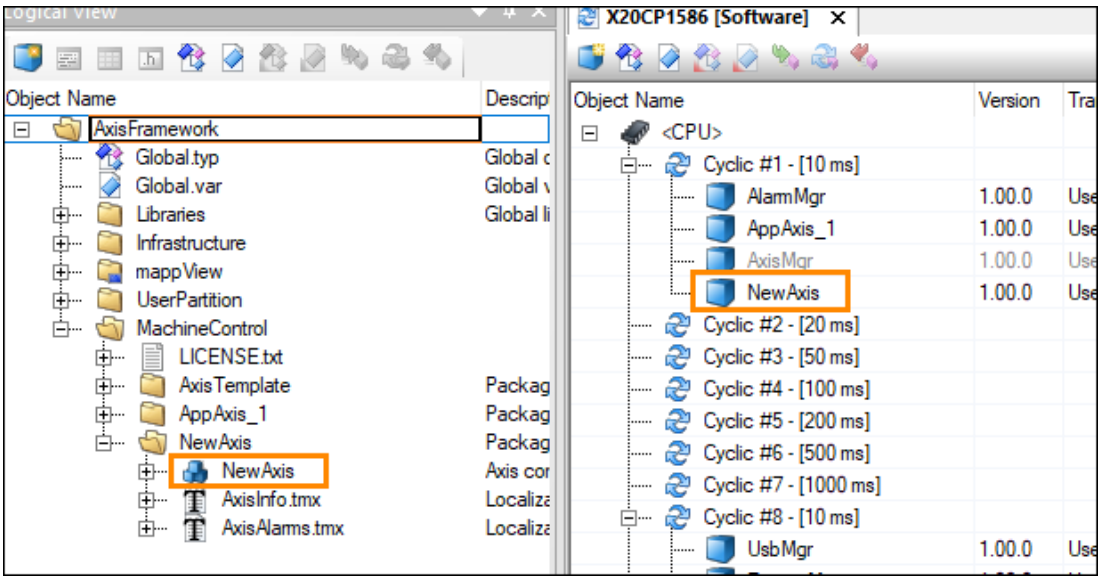
1. Copy the AxisTemplate package that is located in the MachineControl package of the Logical View. Paste it into the MachineControl package.



2. Rename the copied package. (Note: for the rest of these steps, the copied package will be referred to as the "NewAxis" package.)



3. Rename the contained AxisMgr task to match the new package name. Deploy this task to the Software Configuration.



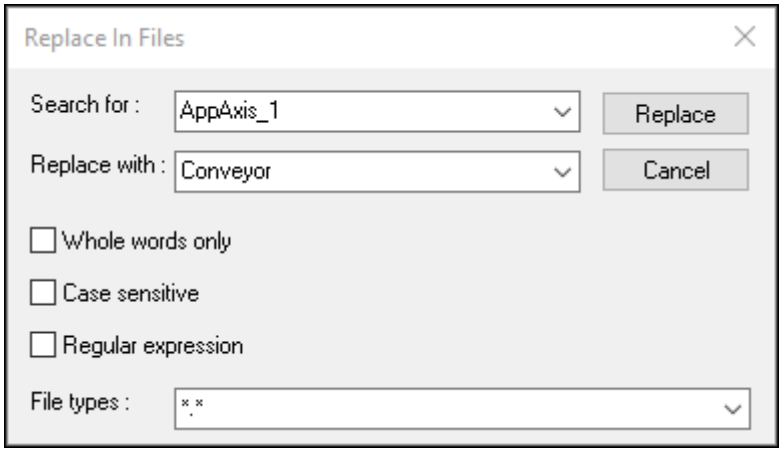
4. In AxisInfo.tmx of the NewAxis package:

    a. Change the namespace to something unique (make sure it starts with "IAT/").

Rename AppAxis_1

Here are the steps to rename the provided AppAxis_1 to something more specific to the application:

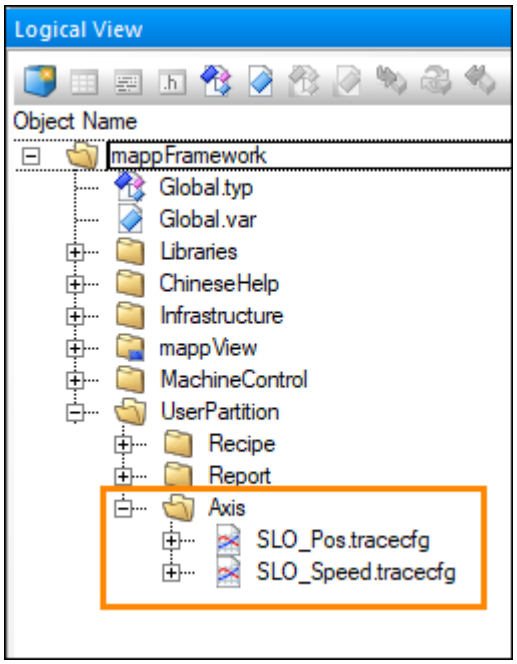
1. Choose a name that is 10 characters or less.
2. Do a replace-in-files (Edit → Find and Replace → Replace in Files) to replace all instances of AppAxis_1 with your new name. Make sure the box for "Whole words only" is NOT checked, and the file types filter is set to "*.*".



3. Manually rename the following files/packages in the Logical View → MachineControl package to your new name:
    a. AppAxis_1 package
    b. AppAxis_1 task
    c. AppAxis_1_Info.tmx
    d. AppAxis_1_Alarms.tmx
4. In the Configuration View → Connectivity → OpcUA, open up Axis.uad in a text editor. On line 36, replace AppAxis_1 with your new name.
5. In the Configuration View → TextSystem, open up TC.textconfig in a text editor. Replace all instances of AppAxis_1 with your new name (4 instances total).

SLO Trace configuration

Two trace configurations are included in the Axis framework - one for position and the other for speed. Since only one trace configuration can be included in the mapp Cockpit package in the Configuration View at a time, these two trace configurations are delivered via the UserPartition\Axis package in the Logical View:



💾 mapp Backup Framework 6

This section describes the **mapp Backup Framework 6**. For full details on **mapp Backup** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

💾 mapp Backup Framework 6 Features

The following features and functionality are included in the **mapp Backup Framework 6**:

• **Easily create and restore a backup** from the **HMI**

• **View a list of all available backups** from the **HMI**

• Choose whether to **automatically back up** the project at a **certain interval**

🔐 Access Rights

The ability to **restore a backup** or **change the automatic backup settings** on the **mapp View HMI** is restricted to the **Administrators** role.

The ability to **delete a backup** is restricted to the **Administrators** or **Service** role.

The default administrative user is **Admin** with default password **123ABc**.
The password **must be changed after import**.

ℹ️ Note

Note that the list of available backups **only updates when the Backup_content is loaded**.

💾 BackupMgr Task Overview

The **BackupMgr** task contains all of the provided Framework code for **mapp Backup**.
This task is located in the **Logical View** within the **Infrastructure package**.

The **BackupMgr** task is deployed to **Task Class 8** by the **Import Tool**.

📄 Task Files

Filename	Type	Description
BackupMgr.st	Main task code	General **mapp function block** handling. This task contains all of the necessary code to **create**, **restore**, and **delete** a backup of the program.
HMIActions.st	Action	Supporting code for **HMI functionality**. Loads and saves the **backup configuration**. Handles the **file manager**.
ChangeConfiguration.st	Action	Contains the programming to modify the **backup configuration at runtime**. Modifications to this action are typically **not necessary**.

⚙️ Localizable Alarm Texts

A **localizable text file** for the **mapp Backup alarms** is included within the **Backup package** (**BackupAlarms.tmx**).

These alarm texts are referenced in the **mapp Backup configuration** instead of the default alarm texts provided by **mapp Services**, which makes it easier for you to expand the texts to include the **language of your choice**.

🔧 Required Modifications

The Framework by default provides a solid foundation, but in order to integrate it fully into your application there are a few modifications that must be made.The following list of modifications are required to get the Framework in a functional state within the application.

> 🚨 **IMPORTANT:**
> The steps on this page must be executed in order to get the Framework into a functional state!

---

1. Change the passwords for the **Admin**, **Operator**, and **Service_Tech** users.
   This is done in the **User.user** file in the Configuration View (**AccessAndSecurity** → **UserRoleSystem** → **User.user**). Note that if you already had users in your project with these same names prior to import, your existing users will remain unchanged and you do not need to update the passwords.
2. The **Backup Framework** uses **retained variables**, which require **nonzero remanent memory**. This must be configured in the **CPU memory configuration** if it is not already configured.
3. If you imported the **mapp View front end** with the Framework:

   a. Assign the **mapp View content**
   (content ID = **Backup_content**) to an area on a page within your visualization.

   b. The ability to **restore a backup** or **change the automatic backup settings** on the **mapp View HMI** is restricted to the **Administrators** role.
   The ability to **delete a backup** is restricted to the **Administrators** or **Service** role.
   Therefore, add a way to **log in on the HMI** (for example, by importing the **mapp UserX** framework).

4. If you did **not** import the mapp View front end with the Framework, connect the **HmiBackup** structure elements to your visualization accordingly.

⚙️ Optional Modifications

The Framework can be adjusted as needed for the application in any way necessary.
This section summarizes some optional modifications that are commonly done to the Framework.

• In the **CPU configuration**, modify the **mappBackupFiles** file device to the desired storage medium.
By default, this corresponds to the **User partition** (**F:\Backup**).

-> If you do, modify or delete lines **10–16** of the **BackupMgr.st INIT** program, which creates the directory **F:\Backup** if it does not already exist.

📂 mapp File Framework 6

This section describes the **mapp File Framework 6**. For full details on **mapp File** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

- Features
- Task Overview
- Required Modification
- Optional Modification
- FIFO

📁 mapp File Framework 6 Features

The following features and functionality are included in the **mapp File Framework 6**:

- **General file explorer functionality** on the **HMI**
  - Create / delete / sort / rename / search / copy / etc
- The ability to **enable a FIFO** for one file device.For more details, see here.

🔐 Access Rights

The ability to **cut / delete a file** and **configure the FIFO** on the **mapp View HMI** is restricted to the **Administrators** or **Service** role.

The default administrative user is **Admin** with default password **123ABc**.
The password **must be changed after import**.

📁 FileMgr Task Overview

The **FileMgr** task contains all of the provided Framework code for **mapp File**.
This task is located in the **Logical View** within the **Infrastructure package**.

The **FileMgr** task is deployed to **Task Class 8** by the **Import Tool**.

📄 Task Files

Filename	Type	Description
FileMgr.st	Main task code	General **mapp function block** handling. Calls all actions.
HMIActions.st	Action	Handles the **file explorer operations** for the **HMI**. Note that full use of this file explorer is **exclusive to administrative users**.
FIFOOperations.st	Action	Handles the implementation of the **FIFO (first-in-first-out)** for the selected file device.

⚙️ Localizable Alarm Texts

A **localizable text file** for the **mapp File alarms** is included within the **File package** (**FileAlarms.tmx**).

These alarm texts are referenced in the **mapp File manager UI configuration** instead of the default alarm texts provided by **mapp Services**, which makes it easier for you to expand the texts to include the **language of your choice**.

🔧 Required Modifications

The Framework by default provides a solid foundation, but in order to integrate it fully into your application there are a few modifications that must be made. The following list of modifications are required to get the Framework in a functional state within the application.

> 🚨 **IMPORTANT:**
> The steps on this page must be executed in order to get the Framework into a functional state!

---

1. Change the passwords for the **Admin**, **Operator**, and **Service_Tech** users. This is done in the **User.user** file in the Configuration View (AccessAndSecurity → UserRoleSystem → User.user).

> Note that if you already had users in your project with these same names prior to import, your existing users will remain unchanged and you do not need to update the passwords.

1. The **File Framework** uses **retained variables**, which require **nonzero remanent memory**. This must be configured in the **CPU memory configuration** if it is not already configured.
2. If you imported the **mapp View front end** with the Framework:
    a. Assign the **mapp View content** (content ID = **File_content**) to an area on a page within your visualization.
    b. The ability to **cut / delete a file** and **configure the FIFO** on the mapp View HMI is restricted to the **Administrators** or **Service** role. Therefore, add a way to **log in on the HMI** (for example, by importing the **mapp UserX** framework).
3. If you did **not** import the mapp View front end with the Framework, connect the **HmiFile** structure elements to your visualization accordingly.

⚙️ Optional Modifications

The Framework can be adjusted as needed for the application in any way necessary. This section summarizes some optional modifications that are commonly done to the Framework.

• N/A

📁 FIFO (First-In-First-Out) Functionality

The **mapp File Framework 6** comes with the option of enabling a **FIFO (first-in-first-out)** for a file device of your choosing.

There are a few key details regarding this feature:

• The **FIFO deletes the oldest files** once the file device starts filling up. There are two configuration options for you to define what **"filling up"** means for your application. A dialog box on the **HMI** allows you to choose between the following:

1. Define a **maximum memory size** and delete the oldest file once the total file device contents exceeds this size.
2. Define a **maximum number of files** and delete the oldest file once the number of files on the file device exceeds this value.

• The user must select **one file device** for the FIFO to be active on.

• Once activated, the FIFO will **check this file device periodically** for size and number of files.

• The FIFO will monitor **all files in the root directory** of the selected file device.Files contained within **any sub-folders will not be checked**.

📝 mapp Recipe Framework 6

This section describes the **mapp Recipe Framework 6**. For full details on **mapp Recipe** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

🗒️ mapp Recipe Framework 6

The following features and functionality are included in the **mapp Recipe Framework 6**:

• The infrastructure to set up **two distinct recipe files**, with a structure variable registered to each

• The ability to **save recipes to a USB drive**

• The ability to **preview and edit a recipe** on the **HMI** before loading it

• The recipe system is set up in the **XML format**. (To convert to a **CSV recipe system**, see here.)

1. Both the **RecipeXML.mprecipexml** and **RecipeCSV.mprecipecsv** configuration files are already included in the Framework to enable the ability to easily switch back and forth. Only **one** of these configuration files is used at a time (i.e. the **MpLink** from only one of these files is referenced in the application).

🔐 Access Rights

The ability to **create / delete / edit a recipe** on the **mapp View HMI** is restricted to the **Administrators** or **Service** roles.

The ability to **load a recipe** is restricted to the **Administrators**, **Service**, or **Operators** roles.

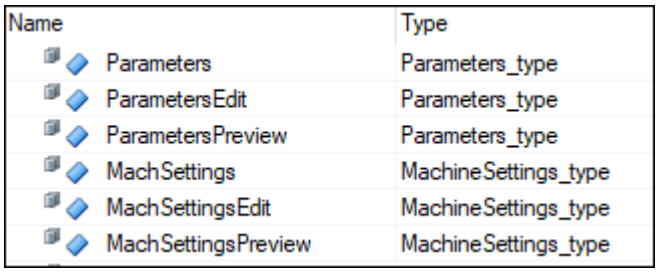
The default administrative user is **Admin** with default password **123ABc**. The password **must be changed after import**.

ℹ️ Note

Note that if you **edit the active recipe**, the **PLC will begin using the updated values immediately**. You do **not** have to perform a **load command** after editing the active recipe.

Recipe System Design

- There are two recipe categories in this recipe system: "Parameters" and "Machine Configuration".
- Each category gets saved to its own recipe file.
- One structure variable is registered to each category.
    - Parameters_type holds the product data.
    - MachineSettings_type holds the machine data.
- Three variables of each datatype are used in order to accomplish the preview functionality and the ability to edit a recipe without formally loading it. For example, let's focus on the parameters structure:
    - The variable Parameters is the actual variable structure that holds the active recipe, which should be used around the application. This variable is not directly registered to the recipe.
    - The variable ParametersPreview is the only variable that is registered to the Product category of the recipe. This allows you to load and preview recipes without actually activating them in the application. If a recipe should be loaded to the application, then the Parameters variable structure gets set equal to the ParametersPreview structure by the Framework.
    - The variable ParametersEdit is used as an intermediary structure to be able to edit the recipe without loading it to the application. It also acts as a buffer between the registered variable so that you can easily discard changes while editing if you need to.



- The custom compound widgets used to display Active and Preview values also incorporate a value compare feature. The widget compares the selected recipe preview values with the active recipe values and if they differ, changes the background of the preview value. This background change is accomplished by changing the style of the standard widget using the modifiedStyle setting under Appearance of the compound widget.

🗒️ mapp Recipe Framework 6 – Task Overview

The **RecipeMgr** task contains all of the provided Framework code for **mapp Recipe**. This task is located in the **Logical View** within the **Infrastructure package**.

The **RecipeMgr** task is deployed to **Task Class 8** by the **Import Tool**.

📄 Task Files

Filename	Type	Description
RecipeMgr.st	Main task code	General **mapp function block** handling. Registers **structure variables** to the two recipes. Loads the **two default recipes**. Calls all actions.
HMIActions.st	Action	Supporting code for **HMI functionality**, such as the **recipe preview** feature. Handles all recipe commands coming from the HMI (**load**, **save**, etc.).
FileOperations.st	Action	**File handling** for copying recipes between the **User partition** and the **USB stick**.

⚙️ Localizable Alarm Texts

A **localizable text file** for the **mapp Recipe alarms** is included within the **Recipe package** (**RecipeAlarms.tmx**).

These alarm texts are referenced in the **mapp Recipe configuration** instead of the default alarm texts provided by **mapp Services**, which makes it easier for you to expand the texts to include the **language of your choice**.

🔧 Required Modifications

The Framework by default provides a solid foundation, but in order to integrate it fully into your application there are a few modifications that must be made.
The following list of modifications are required to get the Framework in a functional state within the application.

> 🚨 **IMPORTANT:**
> The steps on this page must be executed in order to get the Framework into a functional state!

1.**Two default recipes** (**"Machine.mcfg"** for machine settings and **"Default.par"** for product data) must exist in the **mappRecipeFiles** file device at startup.Initial versions of these files are provided for reference in the **Logical View** (**UserPartition** → **Recipe** → **CSVformat** and **UserPartition** → **Recipe** → **XMLformat**).

Only the files that are directly in the **UserPartition** → **Recipe** package will be loaded by the recipe system (which by default is the **XML** file format).

2.Modify the **structure types** within **RecipeMgr.typ** for each registered variable to suit the needs of the application.

- **Parameters_type** is used to hold the **product data**
- **MachineSettings_type** is used to hold the **machine settings / commissioning data**

Further information about the design of the recipe system is provided here.

3.Create new **default recipes** that contain the updated structure types from step **2**.

To do so:

- 1.**Transfer** to the target.
- 2.On the **HMI**, log in as **Admin**.
- 3.On the **Recipe content**, create a **new recipe**. Do not worry about populating values at this step.
- 4.Navigate to the **mappRecipeFiles** file device (by default, **USER_PATH:\Recipe**). Open the newly created recipe in a text editor and edit the **default values** for the recipe parameters

Optional Framework Modifications

The Framework can be adjusted as needed for the application in any way necessary.

This section summarizes some optional modifications that are commonly done to the Framework.

- Change to CSV format
- In the CPU configuration, modify the “mappRecipeFiles” file device to the desired storage medium. By default, this corresponds to the User partition (F:\Recipe).
    - If you do, modify or delete lines 10-16 of the RecipeMgr.st INIT program, which creates the directory F:\Recipe if it does not already exist.

Topics in this section:

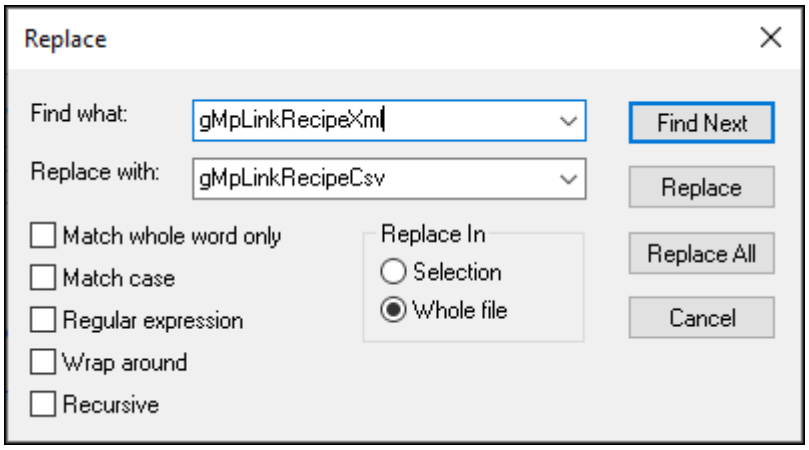
- Change recipe format

Change recipe format

By default the Framework sets up the recipe system in the XML format. The framework includes recipe configurations for both the XML and CSV formats to simplify changeover. If you'd like to switch to CSV.

The steps are as follows:

1. In RecipeMgr.var:
    - Change the datatype of variable **MpRecipeSys** to **MpRecipeCsv**
    - Change the datatype of variable **Header** to **MpRecipeCsvHeaderType**
2. In RecipeMgr.st:
    - Go to Edit → Find and Replace → Replace
    - Replace all instances of **gMpLinkRecipeXml** with **gMpLinkRecipeCsv** in the whole file. There are 8 occurrences total.



3. Delete or move any existing .mcfg and .par files out of the Recipe file device (by default this is F:\Recipe), since these will be the XML format.
4. Copy the default recipe files that are provided in the CSV format (Logical View → UserPartition → Recipe → CSVformat) to the root directory of the Recipe file device. Now the recipe system will load the CSV format versions of the files by default.

📂 mapp UserX Framework 6

This section describes the **mapp UserX Framework 6**. For full details on **mapp UserX** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

👤 mapp UserX Framework 6 Features

The following features and functionality are included in the **mapp UserX Framework 6**:

• **Local user management** (as opposed to **Active Directory**)

• Ability to **import / export user information** via the **HMI**

• Global language and measurement system selectors (if you imported mappView front end)

• The following **predefined roles**:
Everyone, Operators, Service, Administrators

• The following **predefined users**:
User, AdminDefault, Admin, Operator, ServiceTech, SystemUser

🔐 Access Rights

The ability to **view / edit / add / delete / import / export users** in the **UserList widget** on the **mapp View HMI** is restricted to the **Administrators** role.

The default administrative user is **Admin** with default password **123ABc**.
The password **must be changed after import**.

The startup user is **AdminDefault** with the default password **123ABc**. Normally, the startup user is assigned only the Everyone role. However, to allow immediate demonstration of the mapp Framework's functionalities,the startup user is provided with multiple roles by default. Therefore, the startup user **must be changed after import**.

The system user required for backend operations such as importing and exporting users is **SystemUser**, with the hard-coded (in UserXMgr.var file) password **123456ABCdef**. The password **must be changed after import**.
If any automatic logout mechanism is configured, the system user must be logged in again in the UserX Manager program before performing user import/export. If password expiration or compulsory password change is configured, a new password must be provided to the MpUserXLogin FB.
Alternatively, instead of using a dedicated system user, the currently logged-in HMI user can be used. In this case, the username and password must be passed over OPC UA via binding to the MpUserXLogin FB. If this method is used, keep in mind that the current user's password may

👤 UserXMgr Task Overview

The **UserXMgr** task contains all of the provided Framework code for **mapp UserX**. This task is located in the **Logical View** within the **Infrastructure package**.

The **UserXMgr** task is deployed to **Task Class 8** by the **Import Tool**.

📄 Task Files

Filename	Type	Description
UserXMgr.st	Main task code	Handles **user interface interaction**.

⚙️ Localizable Alarm Texts

A **localizable text file** for the **mapp UserX alarms** is included within the **UserX package** (**UserXAlarms.tmx**).

These alarm texts are referenced in the **mapp UserX login configuration** instead of the default alarm texts provided by **mapp Services**, which makes it easier for you to expand the texts to include the **language of your choice**.

🔧 Required Modifications

The Framework by default provides a solid foundation, but in order to integrate it fully into your application there are a few modifications that must be made. The following list of modifications are required to get the Framework in a functional state within the application.

> 🚨 **IMPORTANT:**
> The steps on this page must be executed in order to get the Framework into a functional state!

---

1. Change the passwords for the **User**, **Admin**, **Operator**, **Service_Tech**, and **SystemUser** users. This is done in the **User.user** file in the Configuration View (**AccessAndSecurity** → **UserRoleSystem** → **User.user**).
   Change the password for the **SystemUser** in code as well (**Infrastructure** → **UserX** → **UserXMgr** → **UserXMgr.var**).
   Note that if you already had users in your project with these same names prior to import, your existing users will remain unchanged and you do not need to update the passwords.
2. In the setting of the startup user, choose a user with no administrator rights. This is done in the **Config.mappviewcfg** file in the Configuration View (**mappView** → **Config.mappviewcfg**).
   Then remove or modify the **AdminDefault** user accordingly.
3. If you imported the **mapp View front end** with the Framework, assign the **mapp View content** (content ID = **UserX_content**) to an area on a page within your visualization.
4. If you did **not** import the mapp View front end, connect the **HmiUserX** structure elements to your visualization accordingly.

⚙️ Optional Modifications

The Framework can be adjusted as needed for the application in any way necessary.This section summarizes some optional modifications that are commonly done to the Framework.

Modify the available roles and users if desired.

1. This is done in the **Role.role** and **User.user** files in the **Configuration View**.
2. If you make changes, be sure to update the **UserXCfg.mpuserx** configuration file as well.

   ▪ Note that users only need to be added to the **UserXCfg.mpuserx** configuration file if you want to utilize the **Language**, **Measurement system**, or **Additional Data** properties for that user.
   mapp UserX automatically has access to all users listed in **User.user**.

   ▪ Similarly, note that roles only need to be added to the **UserXCfg.mpuserx** configuration file
   if you want to specify **administrative or access rights**. **mapp UserX** automatically has access to all roles listed in **Role.role**.

In the **CPU configuration**, modify the **mappUserXFiles** file device to the desired storage medium. By default, this corresponds to the **User partition** (**F:\UserX**).

1. If you do, modify or delete lines **10–16** of the **UserXMgr.st INIT** program, which creates the directory **F:\UserX** if it does not already exist.

mapp Audit Framework 6

This section describes the **mapp Audit Framework 6**. For full details on **mapp Audit** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

🛡️ mapp Audit Framework 6 Features

The following features and functionality are included in the mapp Audit Framework 6:

Two audit systems:

- mapp Services events
  - This audit trail captures all built-in events related to the other mapp Services frameworks that are included in the project
- Custom events
  - This audit trail can be used for debugging and troubleshooting the application. The intention with these custom events is that you can leave “breadcrumbs” throughout the application which can help you track down bugs in the application code. For example, if you are troubleshooting a page fault, you could trigger a custom audit event at the top of every task or in several places throughout a task to narrow down where the page fault occurs. The text for the custom audit event is provided directly to the MpAuditCustomEvent() function. For more details, see here.

A mapp View content to view the audit list

- The audit systems use separate display texts, meaning the text shown on the HMI is different than the text that gets exported to the file. This allows you to show a succinct message in the message column on the HMI, and optionally add in the timestamp column / user column / etc to show additional context as needed. Then the exported audit file contains more information directly in the message.

The ability to export an audit archive and choose your export format via the HMI

The ability to configure automatic archiving

The text system has been set up to generate applicable text for all mapp Services audit events

A query along with the supporting state machine to be able to query large amounts of data

---

🛡️ AuditMgr Task Overview

The **AuditMgr** task contains all of the provided Framework code for **mapp Audit**.
This task is located in the **Logical View** within the **Infrastructure package**.

The chart below provides a description of the main task and each action file.

The **AuditMgr** task is deployed to **Task Class 8** by the **Import Tool**.

📄 Task Files

Filename	Type	Description
AuditMgr.st	Main task code	General **mapp function block** handling. Calls all actions. Contains **custom event examples**.
HMIActions.st	Action	Supporting code for **HMI functionality**. Handles **archive settings** and **export commands**. Handles commands to **load and save** an audit configuration. Sets up the **table configuration** for the query table.
ExecuteQuery.st	Action	Executes a **query**. Includes the supporting **state machine** to query large amounts of data.
ChangeConfiguration.st	Action	Contains the programming to modify the **audit configuration at runtime**. Modifications to this action are typically **not necessary**.

🗒️ Text Files in the Audit Package

Several **text files** are provided within the **Audit package**.

🔧 Required Modifications

The Framework by default provides a **solid foundation**, but in order to integrate it fully into your application there are a few modifications that must be made. The following list of modifications is required to get the Framework into a **functional state** within the application.

🚨 **IMPORTANT:**
The steps on this page must be executed in order to get the Framework into a functional state.

---

1. **Change the passwords** for the **Admin**, **Operator**, and **Service_Tech** users.
   This is done in the **User.user** file in the **Configuration View** (**AccessAndSecurity** → **UserRoleSystem** → **User.user**). Note that if you already had users in your project with these same names prior to import,
   your existing users will remain unchanged and you do not need to update the passwords.
2. The **Audit Framework** uses **retained variables**, which require **nonzero remanent memory**. This must be configured in the **CPU memory configuration** if it is not already configured.
3. If you imported the **mapp View front end** with the Framework:
   a. Assign the **mapp View content** (content ID = **Audit_content**) to an area on a page within your visualization.
   b. The ability to **configure automatic archive export** is restricted to the **Administrators** role, and the ability to **create an immediate export** is restricted to the **Administrators** or **Service** role. Therefore, add a way to **log in on the HMI** (for example, by importing the **mapp UserX** framework).
4. If you did **not** import the mapp View front end, connect the **HmiAudit** structure elements to your visualization accordingly (see here for more details).

⚙️ Optional Modifications

The Framework can be adjusted as needed for the application in any way necessary. This section summarizes some optional modifications that are commonly done to the Framework.

- Implement custom audit events throughout the application as needed for debugging during development. These audit events correspond to the custom event audit trail. The full process is as follows:
  a. Increase the value of MAX_CUSTOM_EVENTS within AuditMgr.var according to how many custom events you will use.
  b. Define the text for the type, message, and comment of each custom audit event in the CustomEvent variable structure. This is done in the initialization program of AuditMgr.st starting on line 35. Alternatively, you can do this directly in the "Value" column of AuditMgr.var. These texts will be used when calling the MpAuditCustomEvent() function block. Note that you can only provide the texts in one language (they are not localizable).
  c. Trigger the custom events throughout the application. Examples for how to do this are shown starting on line 75 of AuditMgr.st.
- The localizable text files for mapp Services audit events are provided in the Infrastructure/Audit package of the Logical View. These files were originally taken from the MpAudit library, but they were expanded upon for the framework. Edit the text entries as needed.
- In the CPU configuration, modify the “mappAuditFiles” file device to the desired storage medium. By default, this corresponds to the User partition (F:\Audit).
  a. If you do, modify or delete lines 10-16 of the AuditMgr.st INIT program, which creates the directory F:\Audit if it does not already exist.

📊 mapp Report Framework 6

This section describes the **mapp Report Framework 6**. For full details on **mapp Report** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

📊 mapp Report Framework 6 - Features

The following features and functionality are included in the **mapp Report Framework 6**:

- **Two report configurations**:

Simple Report

```
- Alarm History
- OEE summary
- Temperature sample data
```

Advanced Report, which contains all of the Simple Report information plus:

```
- Audit summary
- A place for a signature
- Batch number information
```

- The ability to **view a list of all available reports** on the **HMI**
- The ability to **create, delete, and view a report** on the **HMI**

🔐 Access Rights

The ability to **delete a report** on the **mapp View HMI** is restricted to the **Administrators** or **Service** role.

The default administrative user is **Admin** with the default password **123ABc**.
The password **must be changed after import**.

📊 ReportMgr Task Overview

The **ReportMgr** task contains all of the provided Framework code for **mapp Report**.
This task is located in the **Logical View** within the **Infrastructure package**.

The chart below provides a description of the main task and each action file.

The **ReportMgr** task is deployed to **Task Class 8** by the **Import Tool**.

📄 Task Files

Filename	Type	Description
ReportMgr.st	Main task code	General **mapp function block** handling. Calls all actions. Populates sample data.
HMIActions.st	Action	Supporting code for **HMI functionality**. Handles the selection between **simple** and **advanced** reports. Manages the **report file explorer**.

⚙️ Localizable Alarm Texts

A **localizable text file** for the **mapp Report alarms** is included within the **Report package** (**ReportAlarms.tmx**).

These alarm texts are referenced in the **mapp Report configuration** instead of the default alarm texts provided by **mapp Services**, which makes it easier to expand the texts to include the **language of your choice**.

🔧 Required Modifications

The Framework by default provides a **solid foundation**, but in order to integrate it fully into your application, a few modifications must be made.

The following list of modifications is required to get the Framework into a **functional state** within the application.

> 🚨 **IMPORTANT:**
> The steps on this page must be executed in order to get the Framework into a functional state.

---

1. Transfer the Report package to the file device

The **UserPartition\Report** package in the **Logical View** must be transferred to the **mappReportFiles** file device (which by default is **USER_PATH:\Report**).

For example, this can be done via an **initial installation**.

⚙️ Optional Modifications

The Framework can be adjusted as needed for the application in any way necessary.

This section summarizes some **optional modifications** that are commonly done to the Framework.

- In the **CPU configuration**, modify the **mappReportFiles** file device to the desired storage medium. By default, this corresponds to the **User partition** (**F:\Report**).
    - If you do, modify or delete lines **10–16** of the **ReportMgr.st INIT** program, which creates the directory **F:\Report** if it does not already exist.

🔁 mapp PackML Framework 6

This section describes the **mapp PackML Framework 6**. For full details on **mapp PackML** itself, see here.

> IMPORTANT: The steps on the "**Required Modifications**" page must be executed in order to get the Framework into a functional state!

Topics in this section:

🔁 mapp PackML Framework 6 Features

The following features and functionality are included in the **mapp PackML Framework 6**:

- Easily view and command **PackML machine states** via **HMI**
- **Basic state machine** containing all **PackML states**, where the user may add machine functionality

🔁 PackMLMgr Task Overview

The **PackMLMgr** task contains all of the provided Framework code for **mapp PackML**.
This task is located in the **Logical View** within the **PackML package**.

The chart below provides a description of the main task and each action file.

📄 Task Files

Filename	Type	Description
PackMLMgr.st	Main task code	General **mapp function block** handling. Calls all actions.
HMIActions.st	Action	Supporting code for **HMI functionality**.
StateMachine_Main.st	Action	Empty **state machine** for the main module of the machine. Contains basic functionality to allow the user to move through **PackML states**. Machine-specific code may be added here.

⚙️ Localizable Alarm Texts

A **localizable text file** for the **mapp PackML alarms** is included within the **PackML package** (**PackMLAlarms.tmx**).

These alarm texts are referenced in the **mapp PackML configuration** instead of the default alarm texts provided by **mapp Services**.
This makes it easier to expand the alarm texts to include the **language of your choice**.

🔧 Required Modifications

The Framework by default provides a **solid foundation**, but in order to integrate it fully into your application, a few modifications must be made.

The following list of modifications is required to get the Framework into a **functional state** within the application.

> 🚨 **IMPORTANT:**
> The steps on this page must be executed in order to get the Framework into a functional state.

---

1. Change default user passwords

Change the passwords for the **Admin**, **Operator**, and **Service_Tech** users.

This is done in the **User.user** file in the **Configuration View** (**AccessAndSecurity** → **UserRoleSystem** → **User.user**).

If users with the same names already existed in the project prior to importing the Framework, those users will remain unchanged and the passwords do not need to be updated.

---

2. Integrate the HMI content

If you imported the **mapp View front end** with the Framework, assign the **mapp View content** (content ID = **PackML_content**) to an area on a page within your visualization.

If you did not import the mapp View front end, connect the **HmiPackML** structure elements to your visualization accordingly (see here for more details).

---

3. Implement the PackML state machine

The action file **StateMachine_Main.st** under **PackMLMgr** contains an **empty state machine** that the user may fill out.

This state machine contains: * Actions for each **PackML state** of the **PackMLMain** module * A line of code indicating which states require a **StateComplete** command to move on to the next state

⚙️ Optional Modifications

The Framework can be adjusted as needed for the application in any way necessary.

This section summarizes some **optional modifications** that are commonly done to the Framework.

In the **CPU configuration**, modify the **mappPackMLFiles** file device to the desired storage medium. By default, this corresponds to the **User partition** (**F:\PackML**).

- If you do, modify or delete lines **10–16** of the **PackML.st INIT** program, which creates the directory **F:\PackML** if it does not already exist.

📌 Topics in this section

- Add additional module

🧩 Adding Additional PackML Modules

By default, the Framework has set up a **single main module** running in **synchronized mode**.

The user may choose to add additional **PackML child modules** by: * Configuring a child module with parent **gPackMLModule_Main** * Using the **StateMachine_Main** action file as a template to create a child state machine * Using the **MpPackMLModule_Main** variable in **PackMLMgr** as a template for calling additional modules

See **Creating a hierarchy** to learn more about hierarchies and configuring child modules.